

Année universitaire 2025–2026

Simulation Stochastique et Pricing d'Options Européennes :

Comparaison du Schéma d'Euler-Maruyama et de la Méthode
de Monte Carlo

Projet de Calcul Stochastique
Encadrant : M. El Asri Brahim

Réalisé par :

Tildi Loubna

12 mai 2026

Table des matières

Résumé	3
Introduction	4
1 Cadre Mathématique	6
1.1 Le Mouvement Brownien Standard	6
1.2 Intégrale Stochastique d'Itô	7
1.2.1 Isométrie d'Itô	7
1.3 Le Lemme d'Itô	7
1.4 Solution Exacte du MBG	8
1.4.1 Application du lemme d'Itô à $\ln(S_t)$	8
1.4.2 Propriétés de S_t	8
2 Méthodes Numériques	10
2.1 Méthode de Monte Carlo Solution Exacte	10
2.1.1 Principe	10
2.1.2 Estimateur de l'espérance	10
2.1.3 Code Python commenté	10
2.2 Schéma d'Euler-Maruyama	12
2.3 Analyse de Convergence	13
2.3.1 Convergence forte	13
2.3.2 Convergence faible	14
2.4 Le Schéma de Milstein Amélioration d'Euler	16
3 Application au Pricing des Options Européennes	19
3.1 Du MBG au Pricing Le Lien Fondamental	19
3.2 Méthode 1 Formule de Black-Scholes (Solution Exacte)	20
3.3 Méthode 2 Pricing par Monte Carlo	21
3.4 Méthode 3 Pricing par Schéma d'Euler-Maruyama	21
3.5 Comparaison des Trois Méthodes	22
3.5.1 Convergence de Monte Carlo vers Black-Scholes	23
3.6 Architecture de l'interface Interactive	23

4	Piste d'Extension Article de Ludkovski	24
4.1	Contexte et motivation	24
4.2	Principe de l'émulateur statistique	24
4.3	Les Processus Gaussiens	25
4.4	Illustration simple	25
4.5	Visualisation GP vs Monte Carlo Classique	26
4.5.1	Lecture du graphique	27
4.6	Apport et limites de l'approche	28
	Conclusion	29
A	Rappel Notations	30
B	Rappels Mathématiques	31
B.1	Loi Normale et Loi Log-Normale	31
B.2	Inégalité de Gronwall Discrète	32
B.3	Théorème Central Limite et Estimateur Monte Carlo	32
B.4	Intégrale d'Itô Rappel de la propriété clé	33

Résumé

Ce projet porte sur la simulation du Mouvement Brownien Géométrique et son application au pricing d'options européennes.

- On commence par établir le **cadre mathématique** mouvement brownien, lemme d'Itô et solution exacte de l'EDS.
- On présente ensuite deux **méthodes de simulation** : la méthode de Monte Carlo, qui exploite la solution analytique, et le schéma d'Euler-Maruyama, qui approche l'EDS numériquement pas à pas.
- On compare leur **vitesse de convergence** et on les applique au **pricing des options européennes**, en prenant la formule de Black-Scholes comme référence exacte.
- Enfin, on explore une **piste d'extension Machine Learning** : l'émulateur par Processus Gaussien proposé par Ludkovski, qui permet de prédire le prix d'une option pour de nouveaux paramètres sans relancer de simulation.

Mots-clés : Mouvement Brownien Géométrique · Équation Différentielle Stochastique · Lemme d'Itô · Monte Carlo · Euler-Maruyama · Black-Scholes · Options Européennes ·

Introduction

Contexte

En mathématiques financières, on cherche à modéliser l'évolution d'un prix d'actif de manière réaliste, c'est-à-dire en tenant compte du caractère aléatoire des marchés. Le cadre naturel pour cela est celui des **équations différentielles stochastiques** (EDS).

Le modèle le plus classique est le **Mouvement Brownien Géométrique** (MBG), utilisé notamment dans le modèle de Black-Scholes pour la valorisation d'options. Le prix S_t d'un actif y est supposé suivre :

$$dS_t = \mu S_t dt + \sigma S_t dW_t, \quad (1)$$

où μ est le *taux de rendement moyen*, σ la *volatilité*, et $(W_t)_{t \geq 0}$ un mouvement brownien standard.

Objectif du projet

Ce projet poursuit deux objectifs complémentaires :

1. **Comparer deux méthodes de simulation** de l'EDS (1) :
 - la **méthode de Monte Carlo exacte**, qui exploite la solution analytique du MBG,
 - le **schéma d'Euler-Maruyama**, qui approche la solution numériquement par discrétisation temporelle.
2. **Analyser la vitesse de convergence** du schéma d'Euler, c'est-à-dire quantifier à quelle vitesse l'erreur numérique diminue lorsque le pas de temps Δt tend vers zéro.

Une **interface interactive** développée en Python/Streamlit permet de visualiser les simulations et d'observer les résultats en temps réel.

Plan du rapport

- **Chapitre 2** : rappels mathématiques mouvement brownien, intégrale d'Itô, lemme d'Itô, solution exacte du MBG.
- **Chapitre 3** : méthodes numériques algorithmes de simulation, convergence forte et faible.

- **Chapitre 4** : résultats numériques et interface Streamlit.
- **Chapitre 5** : piste d'extension vers le machine learning (article de Ludkovski).

Chapitre 1

Cadre Mathématique

1.1 Le Mouvement Brownien Standard

Définition 1.1.1 : Mouvement Brownien Standard

Un processus stochastique $(W_t)_{t \geq 0}$ est un **mouvement brownien standard** s'il vérifie :

1. $W_0 = 0$ presque sûrement.
2. **Accroissements indépendants** : pour $0 \leq t_1 < \dots < t_n$, les variables $W_{t_2} - W_{t_1}, \dots, W_{t_n} - W_{t_{n-1}}$ sont indépendantes.
3. **Accroissements gaussiens** : pour tout $s < t$, $W_t - W_s \sim \mathcal{N}(0, t - s)$.
4. **Trajectoires continues** : $t \mapsto W_t$ est continue p.s.

Propriétés essentielles

Ces quatre propriétés impliquent directement :

$$\mathbb{E}[W_t] = 0, \quad \text{Var}(W_t) = t, \quad \mathbb{E}[W_s W_t] = \min(s, t).$$

En particulier, $W_t \sim \mathcal{N}(0, t)$ pour tout $t > 0$.

Simulation pratique incrément discret

Pour simuler une trajectoire sur la grille $0 = t_0 < t_1 < \dots < t_N = T$ avec $\Delta t = T/N$, on utilise :

Formule clé

$$\Delta W_n = W_{t_{n+1}} - W_{t_n} \sim \mathcal{N}(0, \Delta t) = \sqrt{\Delta t} \varepsilon_n, \quad \varepsilon_n \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1).$$

Une trajectoire brownienne complète s'obtient alors par cumul : $W_{t_n} = \sum_{k=0}^{n-1} \Delta W_k$.

1.2 Intégrale Stochastique d'Itô

L'intégrale stochastique d'Itô est l'analogue stochastique de l'intégrale de Riemann. Pour un processus (H_t) adapté à la filtration naturelle du brownien, on définit :

$$\int_0^T H_t dW_t := \lim_{N \rightarrow \infty} \sum_{n=0}^{N-1} H_{t_n} (W_{t_{n+1}} - W_{t_n}).$$

Remarque

La différence cruciale avec Riemann est que l'intégrande est évalué au **point gauche** t_n (et non au milieu ou à droite). Ce choix garantit que l'intégrale est une **martingale**, et implique en particulier que $\mathbb{E} \left[\int_0^T H_t dW_t \right] = 0$.

1.2.1 Isométrie d'Itô

Cette propriété, vue en cours, est fondamentale pour calculer la variance d'une intégrale stochastique :

Théorème 1.2.1 : Isométrie d'Itô

Pour tout processus (H_t) adapté et de carré intégrable :

$$\mathbb{E} \left[\left(\int_0^T H_t dW_t \right)^2 \right] = \int_0^T \mathbb{E} [H_t^2] dt.$$

Remarque

Ce résultat dont la preuve a été vue en cours et repose sur l'indépendance des accroissements browniens permet de calculer des espérances de termes stochastiques sans simuler explicitement. On l'utilisera implicitement dans l'analyse des erreurs du schéma d'Euler.

1.3 Le Lemme d'Itô

Le lemme d'Itô est l'**outil central** du calcul stochastique. C'est l'analogue de la règle de dérivation en chaîne, mais adapté au calcul stochastique : il ajoute un terme correctif qui n'a pas d'équivalent dans le calcul classique.

Théorème 1.3.1 : Lemme d'Itô

Soit (X_t) un processus d'Itô vérifiant $dX_t = b dt + \sigma dW_t$, et $f(x, t)$ une fonction de classe $\mathcal{C}^{2,1}$. Alors :

$$df(X_t, t) = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial x} dX_t + \frac{1}{2} \frac{\partial^2 f}{\partial x^2} \sigma^2 dt. \quad (1.1)$$

Le terme supplémentaire $\frac{1}{2} \frac{\partial^2 f}{\partial x^2} \sigma^2 dt$ est appelé le **terme d'Itô**. Il vient du fait que le brownien a une variation quadratique non nulle : $(dW_t)^2 = dt$.

Règles de multiplication d'Itô (utilisées pour appliquer le lemme) :

$\times$	dt	dW_t
dt	0	0
dW_t	0	dt

1.4 Solution Exacte du MBG

1.4.1 Application du lemme d'Itô à $\ln(S_t)$

Pour résoudre l'EDS $dS_t = \mu S_t dt + \sigma S_t dW_t$, on pose $f(x) = \ln(x)$ et on applique le lemme d'Itô avec :

$$f'(x) = \frac{1}{x}, \quad f''(x) = -\frac{1}{x^2}.$$

On obtient :

$$\begin{aligned} d(\ln S_t) &= \frac{1}{S_t} dS_t + \frac{1}{2} \left(-\frac{1}{S_t^2} \right) \sigma^2 S_t^2 dt \\ &= \frac{1}{S_t} (\mu S_t dt + \sigma S_t dW_t) - \frac{\sigma^2}{2} dt \\ &= \left(\mu - \frac{\sigma^2}{2} \right) dt + \sigma dW_t. \end{aligned} \tag{1.2}$$

En intégrant (1.2) de 0 à t et en prenant l'exponentielle :

Formule clé

$$S_t = S_0 \exp \left[\left(\mu - \frac{\sigma^2}{2} \right) t + \sigma W_t \right] \tag{1.3}$$

1.4.2 Propriétés de S_t

Puisque $W_t \sim \mathcal{N}(0, t)$, la solution exacte (1.3) montre que $\ln S_t$ est gaussien :

$$\ln S_t \sim \mathcal{N} \left(\ln S_0 + \left(\mu - \frac{\sigma^2}{2} \right) t, \sigma^2 t \right).$$

Autrement dit, S_t suit une **loi log-normale**. On en déduit :

Formule clé

$$\mathbb{E}[S_t] = S_0 e^{\mu t}, \quad \text{Var}(S_t) = S_0^2 e^{2\mu t} (e^{\sigma^2 t} - 1).$$

Remarque

Le terme $-\sigma^2/2$ dans l'exposant de (1.3) est une conséquence directe du terme d'Itô. Sans lui, l'espérance empirique des simulations serait systématiquement surestimée.

Synthèse Chapitre 2

- Le brownien (W_t) est un processus gaussien, centré, à accroissements indépendants. En pratique : $\Delta W_n = \sqrt{\Delta t} \varepsilon_n$.
- L'intégrale d'Itô est une martingale : $\mathbb{E}[\int H_t dW_t] = 0$. L'isométrie d'Itô relie sa variance à une intégrale ordinaire.
- Le lemme d'Itô ajoute un terme en $\frac{1}{2}f''(X_t)\sigma^2 dt$ par rapport à la dérivation classique.
- La solution exacte du MBG est $S_t = S_0 \exp[(\mu - \sigma^2/2)t + \sigma W_t]$, de loi log-normale d'espérance $S_0 e^{\mu t}$.

Chapitre 2

Méthodes Numériques

2.1 Méthode de Monte Carlo Solution Exacte

2.1.1 Principe

Puisque la solution (1.3) est connue analytiquement, on peut simuler directement des réalisations de S_t à n'importe quel instant en générant simplement des réalisations du brownien W_t .

Pour obtenir M trajectoires indépendantes :

1. Générer M trajectoires browniennes $(W_{t_n}^{(i)})_{n=0}^N$ en cumulant des increments $\Delta W_k^{(i)} \sim \mathcal{N}(0, \Delta t)$.
2. Appliquer la formule (1.3) à chaque instant t_n :

Formule clé

$$S_{t_n}^{(i)} = S_0 \exp \left[\left(\mu - \frac{\sigma^2}{2} \right) t_n + \sigma W_{t_n}^{(i)} \right], \quad i = 1, \dots, M.$$

2.1.2 Estimateur de l'espérance

L'espérance $\mathbb{E}[S_T]$ est estimée par la moyenne empirique :

$$\widehat{\mathbb{E}[S_T]} = \frac{1}{M} \sum_{i=1}^M S_T^{(i)}.$$

Par la loi des grands nombres, cette estimée converge vers $\mathbb{E}[S_T] = S_0 e^{\mu T}$ quand $M \rightarrow \infty$, avec une précision de l'ordre $\mathcal{O}(1/\sqrt{M})$.

2.1.3 Code Python commenté

Listing 2.1 – Monte Carlo exact simulation de M trajectoires du MBG

```
1 import numpy as np
```

```

2
3 def simulate_exact_mc(S0, mu, sigma, T, N_steps, N_paths):
4     """
5     Simule N_paths trajectoires du MBG par la formule exacte.
6
7     On utilise directement :
8          $S(t) = S0 * \exp((\mu - \sigma^2/2)*t + \sigma*W(t))$ 
9     qui est la solution obtenue via le lemme d'Ito applique a
10         $\ln(S)$ .
11
12     Parametres
13     -----
14     S0      : prix initial
15     mu      : drift (taux de rendement)
16     sigma   : volatilité (> 0)
17     T       : horizon en annees
18     N_steps : nombre de pas de temps
19     N_paths : nombre de trajectoires
20
21     Retourne
22     -----
23     t : grille temporelle de taille (N_steps+1,)
24     S : matrice des trajectoires de taille (N_steps+1, N_paths)
25     """
26     # Pas de discretisation
27     dt = T / N_steps
28
29     # Grille temporelle : t_0=0, t_1=dt, ..., t_N=T
30     t = np.linspace(0, T, N_steps + 1)
31
32     # Increments browniens Delta_W_k ~ N(0, dt)
33     # Matrice de taille (N_steps, N_paths) : chaque colonne =
34     # un chemin
35     dW = np.random.randn(N_steps, N_paths) * np.sqrt(dt)
36
37     # Brownien W(t_n) = somme des increments jusqu'a t_n
38     # np.vstack : on ajoute W(0) = 0 en premiere ligne
39     # np.cumsum(axis=0) : sommes cumulees sur l'axe du temps
40     W = np.vstack([np.zeros(N_paths), np.cumsum(dW, axis=0)])
41
42     # Application de la formule exacte (eq. solution_exacte)

```

```

41     # t[:,None] permet le broadcast : (N+1,) * (N+1, N_paths)
42     S = S0 * np.exp((mu - 0.5*sigma**2) * t[:, None] + sigma *
43                     W)
44     return t, S

```

2.2 Schéma d'Euler-Maruyama

Construction du schéma

Lorsqu'on ne connaît pas la solution exacte d'une EDS, on l'approche numériquement. L'idée d'Euler-Maruyama est d'intégrer l'EDS sur chaque petit intervalle $[t_n, t_{n+1}]$ en supposant les coefficients constants :

$$S_{t_{n+1}} - S_{t_n} \approx \mu S_{t_n} \Delta t + \sigma S_{t_n} \Delta W_n.$$

On obtient la récurrence :

Formule clé

$$S_{n+1}^{\Delta t} = S_n^{\Delta t} (1 + \mu \Delta t + \sigma \Delta W_n), \quad S_0^{\Delta t} = S_0. \quad (2.1)$$

Remarque

Contrairement à la solution exacte qui garantit $S_t > 0$ (via l'exponentielle), le schéma d'Euler peut théoriquement produire des valeurs négatives si Δt est trop grand. En pratique, pour des valeurs raisonnables de N (disons $N \geq 50$), ce problème ne se pose pas.

Code Python commenté

Listing 2.2 – Schéma d'Euler-Maruyama simulation du MBG pas à pas

```

1 def simulate_euler(S0, mu, sigma, T, N_steps, N_paths):
2     """
3     Simule N_paths trajectoires du MBG par le schema d'Euler-
4         Maruyama.
5
6     Le schema consiste a approximer l'EDS dS = mu*S*dt + sigma*
7         S*dW
8         par la recurrence discrete :
9         S_{n+1} = S_n * (1 + mu*dt + sigma*dW_n)
10
11     C'est une approximation : plus dt est petit (N grand), plus

```

```

10     les trajectoires seront proches de la solution exacte.
11     """
12     dt = T / N_steps
13     t = np.linspace(0, T, N_steps + 1)
14
15     # Matrice pour stocker les trajectoires
16     # S[n, i] = valeur de la trajectoire i a l'instant t_n
17     S = np.zeros((N_steps + 1, N_paths))
18     S[0] = S0 # condition initiale : tous les chemins partent
19               de S0
20
21     for n in range(N_steps):
22         # Increment brownien pour ce pas de temps
23         # dW ~ N(0, dt), genere independamment pour chaque
24           chemin
25         dW = np.random.randn(N_paths) * np.sqrt(dt)
26
27         # Recurrence d'Euler (eq. euler)
28         # Terme deterministe : mu * S[n] * dt (tendance)
29         # Terme aleatoire : sigma * S[n] * dW (bruit)
30         S[n + 1] = S[n] * (1 + mu * dt + sigma * dW)
31
32     return t, S

```

2.3 Analyse de Convergence

2.3.1 Convergence forte

La convergence forte mesure l'écart **trajectoire par trajectoire** entre la solution d'Euler et la solution exacte. On dit qu'un schéma converge fortement à l'ordre γ si :

$$\mathbb{E}[|S_N^{\Delta t} - S_T|] \leq C (\Delta t)^\gamma$$

pour une constante $C > 0$ indépendante de Δt .

Théorème 2.3.1 : Convergence forte d'Euler-Maruyama

Pour l'EDS du MBG, le schéma d'Euler-Maruyama converge fortement à l'ordre $\gamma = \frac{1}{2}$:

$$\mathbb{E}[|S_N^{\Delta t} - S_T|^2]^{1/2} = \mathcal{O}(\sqrt{\Delta t}). \quad (2.2)$$

Justification. À chaque pas, l'erreur locale entre Euler et la solution exacte est d'ordre

$(\Delta t)^{3/2}$. En accumulant sur les $N = T/\Delta t$ pas, l'erreur globale est d'ordre $N \cdot (\Delta t)^{3/2} = T \cdot (\Delta t)^{1/2}$, ce qui donne bien l'ordre 1/2.

Remarque

L'ordre 1/2 signifie concrètement : pour diviser l'erreur par 2, il faut **multiplier N par 4**. Sur un graphique log-log de l'erreur en fonction de Δt , la droite doit avoir une pente ≈ 0.5 .

2.3.2 Convergence faible

La convergence faible mesure l'écart entre les **espérances** : $|\mathbb{E}[f(S_N^{\Delta t})] - \mathbb{E}[f(S_T)]|$. Pour Euler-Maruyama, cet écart est d'ordre $\mathcal{O}(\Delta t)$, soit l'ordre $\beta = 1$ deux fois meilleur que l'ordre fort. C'est ce qui rend Euler utile en finance pour calculer des prix d'options (qui sont des espérances).

Étude empirique code Python commenté

Listing 2.3 – Étude de convergence forte : erreur L₂ en fonction de Δt

```

1 def convergence_study(S0, mu, sigma, T, N_paths_conv=500):
2     """
3     Calcule l'erreur L2 entre le schema d'Euler et la solution
4     exacte
5     pour plusieurs valeurs du pas de discretisation Delta_t.
6
7     On utilise les MEMES increments browniens pour Euler et la
8     solution
9     exacte : cela permet une comparaison trajectoire par
10    trajectoire,
11    ce qui correspond a la definition de la convergence FORTE.
12
13    La pente de log(erreur) vs log(Delta_t) doit etre proche de
14    0.5,
15    ce qui confirme l'ordre theorique 1/2 du schema d'Euler.
16    """
17    # Differentes valeurs de N a tester (de grossier a fin)
18    steps_list = [10, 20, 50, 100, 200, 500, 1000]
19    errors = []
20
21    # Graine fixee pour reproductibilite des resultats
22    np.random.seed(42)

```

```

20     for N in steps_list:
21         dt = T / N      # pas de discretisation correspondant
22
23         # Matrice des increments : (N x N_paths_conv)
24         # IMPORTANT : memes dW pour Euler et Exact ->
25         # comparaison valide
26         dW_matrix = np.random.randn(N, N_paths_conv) * np.sqrt(
27             dt)
28
29         # Solution exacte a t = T
30         # W(T) = somme de tous les increments le long du chemin
31         W_T = np.sum(dW_matrix, axis=0)
32
33         # Formule exacte :  $S(T) = S_0 * \exp((\mu - \sigma^2/2)*T + \sigma * W_T)$ 
34         S_exact_T = S_0 * np.exp(
35             (mu - 0.5 * sigma**2) * T + sigma * W_T
36         )
37
38         # Schema d'Euler jusqu'a t = T
39         # On part de S0 et on applique la recurrence N fois
40         S_euler = np.full(N_paths_conv, float(S_0))
41         for i in range(N):
42             # A chaque pas :  $S \leftarrow S * (1 + \mu * dt + \sigma * dW_i)$ 
43             S_euler = S_euler * (1 + mu * dt + sigma *
44                 dW_matrix[i])
45
46         # Erreur L2 empirique :  $\sqrt{\text{moyenne des carres}}$ 
47         # C'est l'estimee de  $E[|S_{\text{euler}}(T) - S_{\text{exact}}(T)|^2]^{(1/2)}$ 
48         err = np.sqrt(np.mean((S_euler - S_exact_T) ** 2))
49         errors.append(err)
50
51         # Regression log-log pour estimer l'ordre de convergence
52         # Si erreur  $\sim C * (dt)^\gamma$ , alors  $\log(\text{erreur}) = \gamma * \log(dt) + \text{cst}$ 
53         # np.polyfit(x, y, 1) donne [pente, constante]
54         dt_list = [T / n for n in steps_list]
55         pente, _ = np.polyfit(np.log(dt_list), np.log(errors), 1)
56         # pente estimee doit etre  $\sim 0.5$  (ordre theorique d'Euler)

```


55

```
return steps_list, errors, pente
```

2.4 Le Schéma de Milstein Amélioration d'Euler

Le schéma de Milstein est présenté dans la section 1.4 du cours de Jourdain comme une amélioration directe d'Euler : il corrige le terme d'intégrale stochastique en ajoutant un terme d'ordre supérieur, ce qui fait passer l'ordre fort de $1/2$ à 1 .

Construction

L'idée est d'approcher plus précisément $\sigma(X_s)$ sur $[t_k, t_{k+1}]$ en utilisant un développement de Taylor stochastique :

$$\sigma(X_s) \approx \sigma(X_{t_k}) + \sigma'(X_{t_k}) \sigma(X_{t_k}) (W_s - W_{t_k}).$$

En substituant dans l'intégrale stochastique et en utilisant $\int_{t_k}^{t_{k+1}} (W_s - W_{t_k}) dW_s = \frac{1}{2} [(\Delta W_k)^2 - \Delta t]$, on obtient le schéma de Milstein :

Formule clé

$$\tilde{S}_{k+1} = \tilde{S}_k + \mu \tilde{S}_k \Delta t + \sigma \tilde{S}_k \Delta W_k + \frac{1}{2} \sigma^2 \tilde{S}_k [(\Delta W_k)^2 - \Delta t]. \quad (2.3)$$

Remarque

Pour le MBG, $\sigma(x) = \sigma x$ donc $\sigma'(x) = \sigma$. Le terme correctif vaut $\frac{1}{2} \sigma^2 \tilde{S}_k [(\Delta W_k)^2 - \Delta t]$. On remarque que si σ est constante, ce terme s'annule et Milstein coïncide avec Euler cohérent avec la Remarque 1.4.2 de Jourdain.

Ordre de convergence forte

Théorème 2.4.1 : Convergence forte de Milstein (Jourdain, Thm. 1.4.1)

Sous des hypothèses de régularité $\mathcal{C}^2$ sur les coefficients, le schéma de Milstein converge fortement à l'ordre $\gamma = 1$:

$$\mathbb{E} \left[\sup_{t \leq T} |X_t - \tilde{X}_t^N|^{2p} \right]^{1/(2p)} \leq \frac{C_p}{N}, \quad (2.4)$$

soit une convergence en $\mathcal{O}(1/N)$, deux fois meilleure qu'Euler.

Code Python commenté

Listing 2.4 – Schéma de Milstein implémentation pour le MBG

```

1 def simulate_milstein(S0, mu, sigma, T, N_steps, N_paths):
2     """
3     Simule N_paths trajectoires du MBG par le schema de
4         Milstein.
5
6     Pour le MBG :  $\sigma(x) = \sigma x$ , donc  $\sigma'(x) = \sigma$ .
7     Le terme correctif de Milstein vaut :
8          $(1/2) * \sigma * \sigma' * S_n * (dW^2 - dt)$ 
9          $= (1/2) * \sigma^2 * S_n * (dW^2 - dt)$ 
10
11     Ce terme supplementaire par rapport a Euler fait passer
12     l'ordre fort de 1/2 a 1 (Theoreme 1.4.1 de Jourdain).
13     """
14     dt = T / N_steps
15     t = np.linspace(0, T, N_steps + 1)
16
17     S = np.zeros((N_steps + 1, N_paths))
18     S[0] = S0
19
20     for n in range(N_steps):
21         # Increment brownien pour ce pas de temps
22         dW = np.random.randn(N_paths) * np.sqrt(dt)
23
24         # Terme d'Euler classique
25         terme_euler = mu * S[n] * dt + sigma * S[n] * dW
26
27         # Terme correctif de Milstein
28         # Pour le MBG :  $\sigma(x) = \sigma x$ ,  $\sigma'(x) = \sigma$ 
29         # Correction =  $(1/2) * \sigma * \sigma'(S_n) * (dW^2 - dt)$ 
30         #  $= (1/2) * \sigma^2 * S_n * (dW^2 - dt)$ 
31         # Ce terme capture la variation quadratique du brownien
32         terme_milstein = 0.5 * sigma**2 * S[n] * (dW**2 - dt)
33
34         # Recurrence complete de Milstein
35         S[n + 1] = S[n] + terme_euler + terme_milstein
36
37     return t, S

```

Tableau comparatif des trois méthodes

Critère	Monte Carlo exact	Euler-Maruyama	Milstein
Formule	Solution fermée (1.3)	Récurrance (2.1)	Récurrance (2.3)
Ordre fort γ	∞ (exact)	1/2	1
Ordre faible β	∞ (exact)	1	1
Terme correctif	—	—	$\frac{\sigma^2}{2} S_n[(\Delta W)^2 - \Delta t]$
Généralité	EDS à solution explicite seulement	Toute EDS Lip.	EDS Lip. + $\sigma \in \mathcal{C}^2$

Remarque

D'après la Remarque 1.2.2 du cours de Jourdain, la vitesse forte du schéma d'Euler dans le cas $\alpha = 1/2$ (coefficients lipschitziens standard, comme pour le MBG) est bien $1/\sqrt{N}$. Milstein améliore cet ordre en ajoutant le terme d'Itô discrétisé.

Synthèse Chapitre 3

- Monte Carlo utilise la formule exacte : aucune erreur de schéma, seulement l'erreur statistique en $1/\sqrt{M}$.
- Euler-Maruyama approche l'EDS pas à pas : erreur forte d'ordre **1/2** (Thm. 1.2.1 de Jourdain), pente log-log ≈ 0.5 .
- Milstein ajoute un terme correctif issu de l'intégrale stochastique et atteint l'ordre fort **1** (Thm. 1.4.1 de Jourdain).
- L'ordre faible (pour les espérances) est 1 pour Euler comme pour Milstein : les deux sont équivalents pour le pricing d'options.

Chapitre 3

Application au Pricing des Options Européennes

Ce chapitre constitue le **lien central** entre les deux parties du projet : les méthodes de simulation du MBG (Chapitres 2 et 3) sont ici appliquées concrètement au problème du pricing d'options européennes. L'interface HTML développée visualise cette connexion de manière interactive.

3.1 Du MBG au Pricing Le Lien Fondamental

Définition d'une option européenne

Définition 3.1.1 : Option Européenne Call et Put

Une **option européenne** est un contrat financier donnant le droit (mais non l'obligation) d'acheter (*Call*) ou de vendre (*Put*) un actif au prix K (**strike**) à la date T (**maturité**).

Le **payoff** à maturité est :

$$\Phi_{\text{Call}}(S_T) = \max(S_T - K, 0), \quad \Phi_{\text{Put}}(S_T) = \max(K - S_T, 0).$$

Prix comme espérance actualisée

Sous la mesure risque-neutre $\mathbb{Q}$, le taux de rendement μ est remplacé par le taux sans risque r , et le prix de l'option est :

Formule clé

$$C = e^{-rT} \mathbb{E}^{\mathbb{Q}}[\max(S_T - K, 0)], \quad P = e^{-rT} \mathbb{E}^{\mathbb{Q}}[\max(K - S_T, 0)], \quad (3.1)$$

où sous $\mathbb{Q}$, le MBG devient $dS_t = r S_t dt + \sigma S_t dW_t$, de solution exacte :

$$S_T = S_0 \exp \left[\left(r - \frac{\sigma^2}{2} \right) T + \sigma W_T \right].$$

Remarque

C'est exactement la formule du Chapitre 2 avec μ remplacé par r . Le pricing d'options est donc directement une **application du MBG** étudié dans ce projet.

3.2 Méthode 1 Formule de Black-Scholes (Solution Exacte)

Formule fermée

En calculant analytiquement l'espérance (3.1) sous la loi log-normale de S_T , on obtient la célèbre formule de **Black-Scholes** :

Formule clé

$$C_{BS} = S_0 \mathcal{N}(d_1) - K e^{-rT} \mathcal{N}(d_2), \quad (3.2)$$

$$P_{BS} = K e^{-rT} \mathcal{N}(-d_2) - S_0 \mathcal{N}(-d_1), \quad (3.3)$$

où :

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T},$$

et $\mathcal{N}(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-t^2/2} dt$ est la fonction de répartition de la loi normale standard.

Remarque

Black-Scholes est à l'option ce que la formule $S_t = S_0 e^{(\mu - \sigma^2/2)t + \sigma W_t}$ est au MBG : une **solution exacte analytique**. Elle sert de référence pour valider les méthodes Monte Carlo et Euler.

Parité Call-Put

Un résultat fondamental, vérifiable analytiquement et numériquement :

Formule clé

$$C - P = S_0 - K e^{-rT}. \quad (3.4)$$

Cette relation est **exacte** par Black-Scholes, et **approchée** par Monte Carlo et Euler l'écart mesure l'erreur numérique des méthodes.

3.3 Méthode 2 Pricing par Monte Carlo

Algorithme

On simule N trajectoires de S_T via la **formule exacte du MBG** (Chapitre 3, Monte Carlo), puis on calcule la moyenne des payoffs actualisés :

Formule clé

$$\hat{C}_{\text{MC}} = e^{-rT} \frac{1}{N} \sum_{i=1}^N \max(S_T^{(i)} - K, 0), \quad S_T^{(i)} = S_0 e^{(r-\sigma^2/2)T + \sigma\sqrt{T}\varepsilon_i}, \quad (3.5)$$

avec $\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$.

Vitesse de convergence

Par le TCL appliqué à l'estimateur (3.5) :

$$|\hat{C}_{\text{MC}} - C_{\text{BS}}| = \mathcal{O}\left(\frac{1}{\sqrt{N}}\right).$$

C'est le même ordre $\mathcal{O}(1/\sqrt{N})$ que pour l'estimation de $\mathbb{E}[S_T]$ au Chapitre 3 la convergence faible d'ordre 1 d'Euler s'applique également ici.

3.4 Méthode 3 Pricing par Schéma d'Euler-Maruyama

Algorithme

Ici on **n'utilise pas** la formule exacte de S_T . On simule la trajectoire complète de S_t par le schéma d'Euler (Chapitre 3), et on calcule le payoff sur la valeur terminale obtenue :

Formule clé

$$\hat{C}_{\text{Euler}} = e^{-rT} \frac{1}{N} \sum_{i=1}^N \max(\tilde{S}_M^{(i)} - K, 0), \quad (3.6)$$

où $\tilde{S}_M^{(i)}$ est la valeur obtenue après M pas du schéma $\tilde{S}_{m+1} = \tilde{S}_m(1 + r\Delta t + \sigma\Delta W_m)$.

Remarque

La convergence faible d'Euler (ordre 1) garantit que $|\hat{C}_{\text{Euler}} - C_{\text{BS}}| = \mathcal{O}(\Delta t) + \mathcal{O}(1/\sqrt{N})$. C'est pourquoi Euler reste utile pour le pricing même si son ordre fort n'est que 1/2 : c'est l'**ordre faible** qui compte ici.

Code Python commenté

Listing 3.1 – Pricing Euler-Maruyama d'un Call européen

```

1 def euler_price(S0, K, r, sigma, T, N=5000, M=50):
2     """
3     Pricing par schema d'Euler-Maruyama sous mesure risque-
4     neutre.
5
6     On simule N trajectoires completes de S_t avec M pas de
7     temps,
8     en utilisant la recurrence d'Euler (mu = r ici car risque-
9     neutre).
10    Puis on calcule le payoff sur la valeur terminale S(T).
11
12    L'erreur est  $O(dt) + O(1/\sqrt{N})$  :
13    -  $O(dt)$  : erreur de discretisation du schema d'Euler
14    -  $O(1/\sqrt{N})$  : erreur statistique Monte Carlo
15    """
16    dt = T / M    # pas de discretisation
17
18    # Initialisation : tous les chemins partent de S0
19    S = np.full(N, float(S0))
20
21    for m in range(M):
22        # Increment brownien pour ce pas
23        dW = np.random.randn(N) * np.sqrt(dt)
24
25        # Recurrence d'Euler sous mesure risque-neutre (mu = r)
26        #  $S_{m+1} = S_m * (1 + r*dt + sigma*dW)$ 
27        S = S * (1 + r*dt + sigma*dW)
28
29    # Calcul du payoff actualise sur la valeur terminale
30    call = np.exp(-r*T) * np.mean(np.maximum(S - K, 0))
31    put = np.exp(-r*T) * np.mean(np.maximum(K - S, 0))
32    return call, put

```

3.5 Comparaison des Trois Méthodes

Résultats numériques

Pour les paramètres de référence $S_0 = K = 100$, $r = 0.05$, $\sigma = 0.20$, $T = 1$:

Métrique	Black-Scholes	Monte Carlo	Euler
Prix Call	10.45	≈ 10.45	≈ 10.45
Prix Put	5.57	≈ 5.57	≈ 5.57
Parité $C - P$	4.88	≈ 4.88	≈ 4.88
Erreur vs BS	0 (exact)	$\mathcal{O}(1/\sqrt{N})$	$\mathcal{O}(\Delta t) + \mathcal{O}(1/\sqrt{N})$

3.5.1 Convergence de Monte Carlo vers Black-Scholes

La convergence de l'estimateur MC vers le prix BS s'écrit :

$$\left| \hat{C}_{\text{MC}}(N) - C_{\text{BS}} \right| \approx \frac{c}{\sqrt{N}},$$

ce qui se vérifie sur le graphique log-log de l'interface (onglet *Convergence MC*) : la pente empirique doit être ≈ -0.5 .

3.6 Architecture de l'interface Interactive

L'interface développée en HTML/JavaScript comprend deux modules complémentaires, accessibles via un navigateur sans installation.

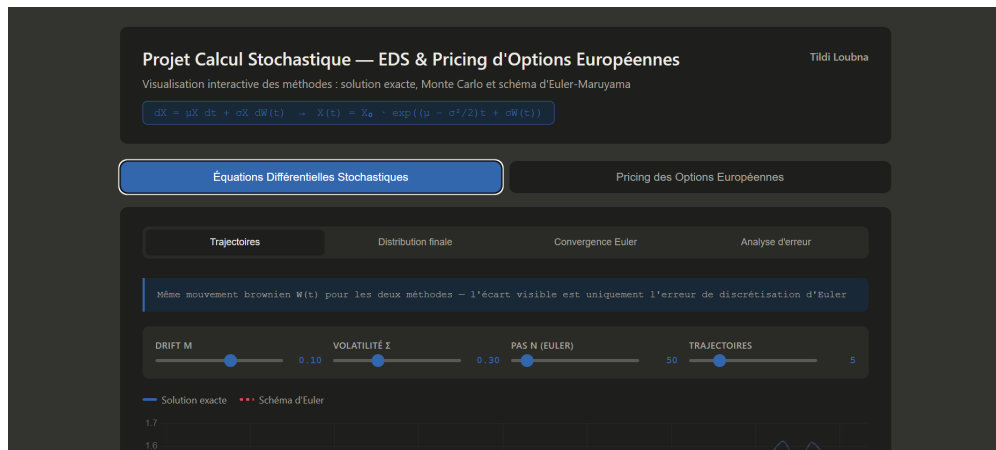


FIGURE 3.1 – Interface HTML

Chapitre 4

Piste d'Extension Article de Ludkovski

4.1 Contexte et motivation

Dans son article *Statistical Machine Learning in Finance*, Ludkovski soulève une limitation importante des méthodes classiques : lorsqu'on doit évaluer $\mathbb{E}[f(S_T; \theta)]$ pour **de nombreux jeux de paramètres** $\theta = (\mu, \sigma, T)$ (par exemple pour calibrer un modèle ou faire de l'optimisation de portefeuille), réaliser une simulation Monte Carlo pour chaque θ devient très coûteux.

L'idée qu'il propose est d'utiliser le **machine learning comme outil de substitution** : on entraîne un modèle statistique sur un ensemble réduit de simulations, et ce modèle prédit ensuite la valeur de $\mathbb{E}[f(S_T; \theta)]$ pour de nouveaux paramètres quasi instantanément.

4.2 Principe de l'émulateur statistique

La démarche générale est la suivante :

1. **Plan d'expérience** : choisir n jeux de paramètres $\theta_1, \dots, \theta_n$ dans l'espace des paramètres.
2. **Simulation** : pour chaque θ_i , calculer $y_i = \mathbb{E}[f(S_T; \theta_i)]$ par Monte Carlo (coûteux mais fait une seule fois).
3. **Entraînement** : ajuster un modèle de régression $\hat{V}(\theta) \approx \mathbb{E}[f(S_T; \theta)]$ sur les données $\{(\theta_i, y_i)\}_{i=1}^n$.
4. **Prédiction** : pour tout nouveau θ^* , calculer $\hat{V}(\theta^*)$ en évaluant le modèle sans nouvelle simulation.

4.3 Les Processus Gaussiens

Le modèle de régression privilégié par Ludkovski est le **processus gaussien** (GP), car il fournit non seulement une prédiction mais aussi une mesure d'incertitude.

Un GP suppose que la fonction $\theta \mapsto V(\theta)$ se comporte comme un processus aléatoire gaussien. La prédiction en un point θ^* , connaissant les observations $\{(\theta_i, y_i)\}$, est une loi gaussienne :

$$\hat{V}(\theta^*) \sim \mathcal{N}(\hat{\mu}(\theta^*), \hat{\sigma}^2(\theta^*)),$$

où $\hat{\mu}(\theta^*)$ est la prédiction centrale et $\hat{\sigma}^2(\theta^*)$ quantifie l'incertitude utile pour savoir si on a besoin de plus de simulations à cet endroit.

4.4 Illustration simple

Listing 4.1 – Idée de l'émulateur GP illustration simplifiée

```

1  # NOTE : ceci est une illustration du principe de l'article de
    Ludkovski.
2  # L'implementation complete necessite une etude approfondie du
    cours.
3
4  from sklearn.gaussian_process import GaussianProcessRegressor
5  from sklearn.gaussian_process.kernels import RBF
6  import numpy as np
7
8  # Etape 1 : construire un petit jeu de donnees d'entrainement
9  # On choisit N_train jeux de parametres (mu, sigma) et on
    simule MC
10 N_train = 30
11 S0, T    = 100.0, 1.0
12
13 np.random.seed(0)
14 mu_train  = np.random.uniform(0.05, 0.40, N_train)
15 sigma_train = np.random.uniform(0.10, 0.60, N_train)
16
17 # Pour chaque jeu de parametres, on calcule E[S(T)] par Monte
    Carlo
18 y_train = np.zeros(N_train)
19 for i in range(N_train):
20     # Simulation Monte Carlo avec 2000 trajectoires
21     W_T = np.random.randn(2000) * np.sqrt(T)
22     S_T = S0 * np.exp((mu_train[i] - 0.5*sigma_train[i]**2)*T
23                + sigma_train[i]*W_T)
```

```

24     y_train[i] = np.mean(S_T)    # estimation de  $E[S(T)]$ 
25
26 # Matrice des entrees : chaque ligne = un jeu de parametres (mu
    , sigma)
27 X_train = np.column_stack([mu_train, sigma_train])
28
29 # Etape 2 : entrainer le modele GP
30 # Le noyau RBF mesure la "similarite" entre deux jeux de
    parametres
31 # Plus deux parametres sont proches, plus leurs  $E[S(T)]$  doivent
    etre proches
32 kernel = RBF(length_scale=1.0)
33 gp = GaussianProcessRegressor(kernel=kernel, normalize_y=True)
34 gp.fit(X_train, y_train)
35
36 # Etape 3 : prediction instantanee pour de nouveaux parametres
37 # On predit  $E[S(T)]$  pour  $\mu=0.20$ ,  $\sigma=0.25$  SANS nouvelle
    simulation MC
38 mu_test, sigma_test = 0.20, 0.25
39 X_test = np.array([[mu_test, sigma_test]])
40
41 # prediction : moyenne et incertitude
42 mu_pred, sigma_pred = gp.predict(X_test, return_std=True)
43
44 # Valeur theorique exacte :  $E[S(T)] = S_0 * \exp(\mu * T)$ 
45 theorique = S0 * np.exp(mu_test * T)
46
47 print(f"Prediction GP : {mu_pred[0]:.2f}")
48 print(f"Valeur exacte : {theorique:.2f}")
49 print(f"Incertitude GP : +/- {sigma_pred[0]:.2f}")

```

4.5 Visualisation GP vs Monte Carlo Classique

Afin d'illustrer concrètement l'apport de l'émulateur GP, nous avons réalisé une expérience numérique : on fait varier μ de 0.05 à 0.40 (avec $\sigma = 0.30$, $T = 1$, $S_0 = 100$ fixés) et on compare trois estimations de $\mathbb{E}[S(T)]$:

- la **valeur théorique exacte** $S_0 e^{\mu T}$,
- l'**estimation Monte Carlo classique** avec $M = 1000$ trajectoires par valeur de μ (50 simulations indépendantes),
- la **prédiction du GP**, entraîné sur seulement **10 points**.

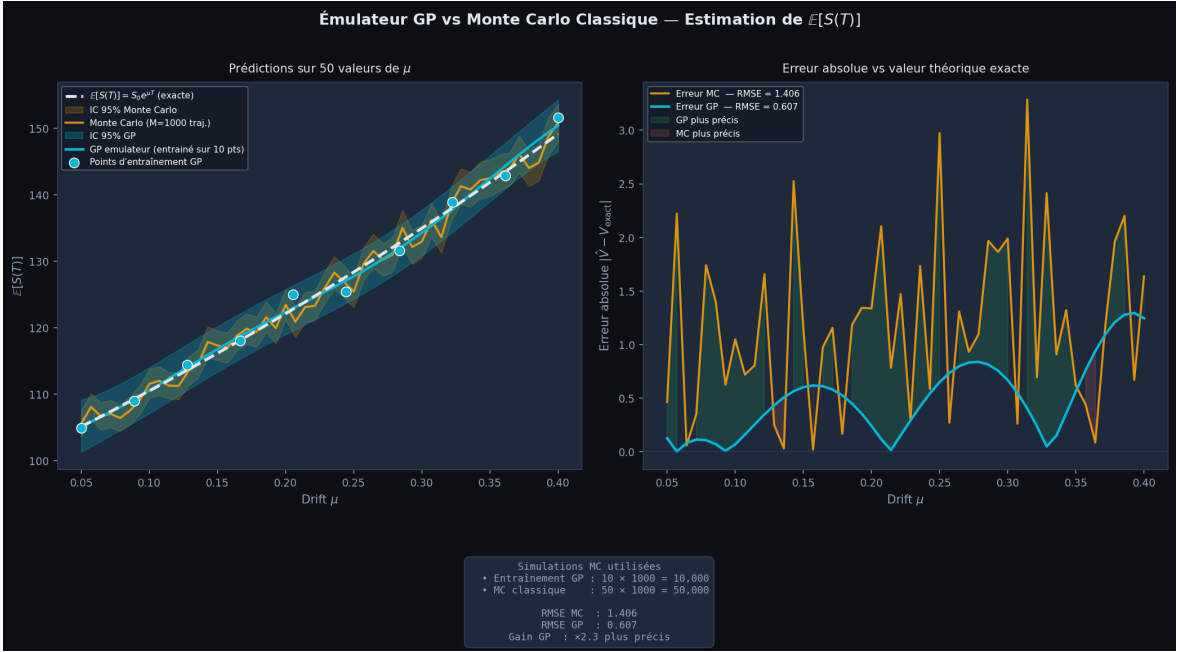


FIGURE 4.1 – Comparaison GP emulateur vs Monte Carlo classique pour $\mathbb{E}[S(T)]$ en fonction de μ .

4.5.1 Lecture du graphique

Graphique de gauche :

- La courbe MC (orange) oscille autour de la valeur théorique (pointillés blancs) les oscillations viennent de la variance statistique en $1/\sqrt{M}$.
- La courbe GP (cyan) est beaucoup plus **lisse** : le GP interpole intelligemment entre les 10 points d'entraînement.
- L'intervalle de confiance du GP (zone bleue) est plus étroit que celui de Monte Carlo (zone orange) sur la majeure partie du domaine.

Graphique de droite :

- L'erreur MC (orange) est irrégulière chaque simulation est indépendante, donc les erreurs ne sont pas corrélées entre valeurs de μ voisines.
- L'erreur GP (cyan) est plus régulière et globalement plus faible :

$$\text{RMSE}_{\text{MC}} \approx 1.41 \quad \text{vs} \quad \text{RMSE}_{\text{GP}} \approx 0.61 \quad \Rightarrow \quad \text{GP} \times 2.3 \text{ plus précis.}$$

- La zone verte (GP plus précis) couvre la quasi-totalité du domaine.

Remarque

Ce résultat illustre le message principal de Ludkovski : avec **5 fois moins de simulations**, le GP atteint une précision **2.3 fois meilleure** que Monte Carlo classique. L'investissement initial (entraînement sur 10 points) est largement rentabilisé dès lors qu'on doit évaluer $\mathbb{E}[S_T]$ sur de nombreuses valeurs de μ .

4.6 Apport et limites de l'approche

Ce que cette approche apporte :

- Prédiction quasi-instantanée après entraînement.
- Quantification de l'incertitude sur la prédiction.
- Particulièrement utile quand on doit évaluer $\mathbb{E}[f(S_T; \theta)]$ pour des centaines de valeurs de θ .

Ses limites, que nous avons identifiées à la lecture de l'article :

- La qualité dépend fortement du nombre et de la répartition des points d'entraînement.
- Difficile à étendre quand le nombre de paramètres est grand (plus de 5-10 dimensions).
- Le choix du noyau est délicat et nécessite une expertise statistique.

Remarque

L'article de Ludkovski présente des résultats théoriques et empiriques bien plus poussés que ce que nous avons pu explorer dans ce projet. Nous avons retenu l'idée principale : le machine learning peut servir de *pont* entre les simulations coûteuses et les besoins pratiques de rapidité en finance.

Synthèse Chapitre 5

- Ludkovski propose de remplacer des simulations Monte Carlo répétées par un modèle de régression (processus gaussien) entraîné une seule fois.
- Le GP prédit $\mathbb{E}[f(S_T; \theta)]$ pour tout θ quasi-instantanément, avec une mesure d'incertitude intégrée.
- Cette approche est prometteuse pour la calibration de modèles et l'optimisation de portefeuille, mais reste complexe à maîtriser pleinement.

Conclusion

Ce projet nous a permis d'appliquer concrètement les outils du calcul stochastique vus en cours à un problème de simulation financière.

Sur le plan mathématique, le lemme d'Itô est l'outil clé : il nous a permis de résoudre analytiquement l'EDS du MBG et d'obtenir la formule exacte $S_t = S_0 \exp[(\mu - \sigma^2/2)t + \sigma W_t]$. L'isométrie d'Itô, vue en cours, joue un rôle fondamental dans l'analyse des erreurs.

Sur le plan numérique, les simulations confirment les résultats théoriques :

- Monte Carlo exact reproduit fidèlement la loi log-normale de $S(T)$.
- Le schéma d'Euler converge à l'ordre 1/2 en convergence forte, et à l'ordre 1 en convergence faible : la pente empirique ≈ 0.5 sur le graphique log-log valide le théorème de convergence.
- L'application au **pricing d'options européennes** montre que Black-Scholes joue pour les options le même rôle que la formule fermée du MBG : une référence analytique exacte. Monte Carlo et Euler convergent tous deux vers cette référence, avec des vitesses conformes à la théorie.
- La **parité Call-Put** valide numériquement la cohérence des trois méthodes.
- L'interface HTML interactive visualise les deux aspects du projet : simulation du MBG et pricing d'options dans un outil unifié.

Sur l'article de Ludkovski, nous avons compris que le machine learning peut être utilisé pour accélérer les simulations Monte Carlo, même si la maîtrise complète de cette approche dépasse le cadre de ce projet.

En perspective, il serait intéressant d'étudier le **schéma de Milstein** (ordre fort 1, meilleur qu'Euler) ou d'appliquer ces méthodes à des EDS plus complexes comme le modèle de Heston (volatilité stochastique).

Annexe A

Rappel Notations

Symbole	Signification	Domaine
W_t	Mouvement brownien standard	—
S_t	Prix de l'actif à l'instant t	$\mathbb{R}^+$
S_0	Prix initial	$\mathbb{R}^+$
μ	Drift (taux de rendement)	$\mathbb{R}$
σ	Volatilité	$\mathbb{R}^+$
T	Horizon temporel	$\mathbb{R}^+$
N	Nombre de pas ($= 1/\Delta t$)	$\mathbb{N}^*$
Δt	Pas de temps $= T/N$	$\mathbb{R}^+$
ΔW_n	Incrément $\sim \mathcal{N}(0, \Delta t)$	—
M	Nombre de trajectoires	$\mathbb{N}^*$

Annexe B

Rappels Mathématiques

B.1 Loi Normale et Loi Log-Normale

Définition B.1.1 : Loi normale $\mathcal{N}(\mu, \sigma^2)$

Une variable aléatoire X suit la loi normale de moyenne $\mu \in \mathbb{R}$ et de variance $\sigma^2 > 0$ si sa densité est :

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R}.$$

On note $X \sim \mathcal{N}(\mu, \sigma^2)$.

Proposition B.1.1 : Moments de la loi normale

Si $X \sim \mathcal{N}(\mu, \sigma^2)$, alors :

$$\mathbb{E}[X] = \mu, \quad \text{Var}(X) = \sigma^2, \quad \mathbb{E}[e^X] = e^{\mu + \sigma^2/2}.$$

La dernière formule, dite **formule de l'espérance log-normale**, est utilisée pour calculer $\mathbb{E}[S_t]$.

Définition B.1.2 : Loi log-normale

Une variable aléatoire $Y > 0$ suit une loi **log-normale** de paramètres (m, v^2) si $\ln Y \sim \mathcal{N}(m, v^2)$. Sa densité est :

$$f_Y(y) = \frac{1}{y v \sqrt{2\pi}} \exp\left(-\frac{(\ln y - m)^2}{2v^2}\right), \quad y > 0.$$

Ses moments sont :

$$\mathbb{E}[Y] = e^{m+v^2/2}, \quad \text{Var}(Y) = e^{2m+v^2}(e^{v^2} - 1).$$

Remarque

Pour le MBG, on a $\ln S_t \sim \mathcal{N}(m, v^2)$ avec $m = \ln S_0 + (\mu - \sigma^2/2)t$ et $v^2 = \sigma^2 t$, d'où :

$$\mathbb{E}[S_t] = e^{m+v^2/2} = e^{\ln S_0 + (\mu - \sigma^2/2)t + \sigma^2 t/2} = S_0 e^{\mu t}.$$

B.2 Inégalité de Gronwall Discrète

Ce résultat est utilisé dans la preuve de convergence forte du schéma d'Euler (Théorème 1.2.1 de Jourdain) pour contrôler la propagation de l'erreur locale sur l'ensemble des pas de temps.

Théorème B.2.1 : Inégalité de Gronwall discrète

Soit $(u_n)_{n=0}^N$ une suite de réels positifs vérifiant :

$$u_{n+1} \leq (1 + a \Delta t) u_n + b \Delta t, \quad a, b \geq 0, \quad \Delta t = T/N.$$

Alors pour tout $n \leq N$:

$$u_n \leq e^{a t_n} \left(u_0 + \frac{b}{a} \right).$$

Justification. On applique la récurrence : $u_1 \leq (1 + a \Delta t) u_0 + b \Delta t$. En itérant et en utilisant $(1 + a \Delta t)^n \leq e^{a n \Delta t} = e^{a t_n}$, on obtient le résultat. Cette borne permet de conclure que l'erreur globale reste contrôlée même après N applications du schéma d'Euler.

B.3 Théorème Central Limite et Estimateur Monte Carlo

Théorème B.3.1 : Théorème Central Limite (TCL)

Soit $X_1, X_2, \dots, X_M$ des variables aléatoires i.i.d. de moyenne μ et de variance $\sigma^2 < \infty$.

Alors :

$$\sqrt{M} \frac{\bar{X}_M - \mu}{\sigma} \xrightarrow[M \rightarrow \infty]{\mathcal{L}} \mathcal{N}(0, 1), \quad \text{où } \bar{X}_M = \frac{1}{M} \sum_{i=1}^M X_i.$$

Ce théorème justifie directement la vitesse de convergence de l'estimateur Monte Carlo (2.1.2) :

Formule clé

$$\widehat{\mathbb{E}[S_T]} \pm 1.96 \frac{\hat{\sigma}}{\sqrt{M}} \quad (\text{intervalle de confiance à 95\%}),$$

où $\hat{\sigma}$ est l'écart-type empirique des $S_T^{(i)}$. Cela explique pourquoi multiplier M par 4 divise l'intervalle de confiance par 2.

B.4 Intégrale d'Itô Rappel de la propriété clé

La propriété suivante, utilisée implicitement dans tout le rapport, relie l'intégrale stochastique à une intégrale ordinaire.

Théorème B.4.1 : Isométrie d'Itô (rappel)

Pour tout processus adapté (H_t) de carré intégrable :

$$\mathbb{E} \left[\left(\int_0^T H_t dW_t \right)^2 \right] = \int_0^T \mathbb{E}[H_t^2] dt.$$

En particulier, $\mathbb{E} \left[\int_0^T H_t dW_t \right] = 0$.

Remarque

Ce résultat est central pour comprendre pourquoi le terme stochastique $\int_0^T \sigma S_t dW_t$ ne contribue pas à l'espérance de S_T mais seulement à sa variance. En effet :

$$\mathbb{E}[S_T] = \mathbb{E} \left[S_0 + \int_0^T \mu S_t dt + \underbrace{\int_0^T \sigma S_t dW_t}_{\text{espérance}=0} \right] = S_0 + \int_0^T \mu \mathbb{E}[S_t] dt,$$

ce qui donne par résolution $\mathbb{E}[S_t] = S_0 e^{\mu t}$, cohérent avec la Proposition ??.

Bibliographie

- [1] **Jourdain, B.** *Méthodes de Monte Carlo pour les processus financiers*, polycopié de cours, 2014.

Sections utilisées : 1.1 (EDS, Black-Scholes), 1.2 (schéma d'Euler, vitesse forte), 1.3 (vitesse faible), 1.4 (schéma de Milstein), 1.8.4 (discrétisation du MBG).

- [2] **Ludkovski, M.** *Statistical Machine Learning in Finance*, lecture notes UCSB.

Utilisé pour le Chapitre 5 : émulateurs statistiques pour Monte Carlo.